

The ablation story (Configs A, B, C) is what makes this publishable beyond just "fine-tuning helps." Config B vs Config A specifically isolates the value of your synthetic data generation — that's a standalone contribution.

The winning scenario the prompt sets up: if fine-tuned IndicWav2Vec beats Voxtral on banking WER (46.01% → ?) while being 6× faster (RTF 0.09 vs 0.54), your production recommendation writes itself. That's the krim.ai deployment answer and the thesis conclusion in one result.



Finetuning master prompt

Document · MD



Download

my mentor said to focus on various accents of hindi also, like in north india hindi from punjab, haryana, bihar each sounds different, since we are making an autonomous banking system for indian users which supports back office operations such as debt collection, loan journey, customer onboarding, how should i study about these in asr? he also told me to look into Vaani project (it some voice/audio data collection)

Synthesized Hindi accent linguistics, datasets, and ASR methodologies for banking applications >

This search turned up something really valuable — there are now three distinct resources that perfectly map to your use case, and they weren't all in the survey paper you had. Let me break everything down properly.

First — why Hindi accents are a genuine ASR problem, not just a nice-to-have

Hindi is spoken as a **second or third language** by most of India. A customer calling a bank in Delhi might be from Bihar (Bhojpuri substrate), Punjab (Punjabi substrate), or Tamil Nadu (Dravidian substrate). Each brings completely different phonological interference:

Region	Native Language	How it affects Hindi speech
Punjab	Punjabi	Aspirated consonants reduced — "phir" sounds like "fir", retroflex sounds softened

Region	Native Language	How it affects Hindi speech
Haryana	Haryanvi	Heavy retroflex consonants, elongated vowels, "का" becomes "कै"
Bihar/UP	Bhojpuri/Awadhi	"व" → "ब" substitution ("vaad" → "baad"), nasalization, different tonal patterns
Bengal	Bengali	"ঞ" → "ঞ" confusion, vowel length differences, "loan" → "loon"
South India	Tamil/Telugu/Kannada	Retroflex consonants hypercorrected, gemination, no aspiration distinction
Maharashtra	Marathi	Dental-retroflex confusion, different stress patterns

LAHAJA's fine-grained analysis shows performance declines specifically for speakers from North-East and South India, especially with content heavy in named entities and specialized terminology — exactly what banking calls contain. This is not an edge case — it's the mainstream population you're serving. [arxiv](#)

Project Vaani — what it actually is

Project Vaani, by IISc Bangalore and ART PARK, is one of the largest datasets of Indian dialects ever built. Upon completion it will contain more than 150,000 hours of audio across all districts in India — with data from about 1 million people across all 773 districts open-sourced. [Hugging Face](#)

Following rigorous multi-stage quality evaluation including both automated and manual checks, they have open-sourced around 289K images, approximately 31,270 hours of audio recordings, and around 2,067 hours of transcribed speech, encompassing 109 languages from 165 districts across 31 states and union territories.

[arXiv](#)

The key practical detail: approximately 10% of the Vaani data is transcribed. Untranscribed speech data can be used for self-supervised speech representation learning and for applications such as language identification and accent classification with accurate language and accent labels. [arXiv](#)

For your use case specifically — there's already a real-world proof point. SandLogic fine-tuned an ASR model using the Hindi subset of the Vaani dataset, carefully filtering 366 hours of clean, aligned Hindi audio-text pairs covering 8 states and 51 districts. The model achieved a ~21% relative WER reduction on the LAHAJA

benchmark, and in real-world deployment reduced WER by ~55% for a healthcare client and ~47% for a digital services client — directly proving that Vaani-based fine-tuning transfers to call center performance. [lith](#)

How to access it:

```
python

from datasets import load_dataset

# District-level subsets – load specific Hindi-belt districts
# Bihar districts: Patna, Gaya, Muzaffarpur
dataset = load_dataset(
    "ARTPARK-IISc/VAANI",
    "Bihar_Patna",          # format: State_District
    token=<hf_token>        # requires HuggingFace login
)

# For accent study: load multiple districts and compare phoneme distributions
hindi_belt_districts = [
    "Bihar_Patna",
    "Bihar_Gaya",
    "UttarPradesh_Lucknow",
    "Punjab_Ludhiana",
    "Haryana_Rohtak",
    "Rajasthan_Jaipur",
]
```

The district-level metadata is the gold — data can be filtered and downloaded district-wise from their platform, which means you can build accent-stratified training sets precisely targeting the Hindi-belt states where your banking customers are concentrated. [arXiv](#)

LAHAJA — your accent evaluation benchmark

LAHAJA contains read and extempore speech on a diverse set of topics and use cases, with a total of 12.5 hours of Hindi audio sourced from 132 speakers spanning 83 districts of India. [ResearchGate](#)

This is the benchmark your mentor was pointing at. Two things that make it perfect for your work:

1. It has banking-specific speaker metadata. The metadata CSV contains occupation domain including Financial Services (Banking and Insurance) as a speaker category —

you can filter for banking-domain speakers specifically. [arxiv](#)

2. It focuses on non-native Hindi. The primary languages of non-native speakers covered 19 of the 22 scheduled languages of India, spanning Indo-Aryan, Dravidian, and Tibeto-Burman language families — with 65 males and 67 females across age groups 18–30, 30–45, 45–60, and 60+. [arxiv](#)

This is exactly your customer base — people who speak Hindi as a second language when calling a bank.

Get it:

```
bash
```

```
git clone https://github.com/AI4Bharat/Lahaja  
# CSV has per-speaker accent metadata – use this to stratify evaluation
```

RESPIN — the dataset you didn't know existed (and it has a finance domain)

This is the most exciting find from the search. RESPIN-S1.0 is the largest publicly available dialect-rich read-speech corpus for Indian languages, comprising more than 10,000 hours of validated audio across nine languages including Hindi, Bengali, Bhojpuri, Chhattisgarhi, Magahi, and Maithili — covering more than 38 dialects across two high-impact domains: agriculture and **finance**. [arxiv](#)

RESPIN provides dialect-aware phonetic lexicons, speaker metadata, and reproducible train/dev/test splits. Fine-tuning on RESPIN significantly improves recognition accuracy over pretrained baselines across TDNN-HMM, E-Branchformer, Whisper, and wav2vec2-based models. [Semantic Scholar](#)

This is uniquely valuable for you: the finance domain subset directly overlaps with banking terminology in Hindi dialects. Bhojpuri, Magahi, and Maithili are the dominant substrate languages of Bihar — the exact accent interference your debt collection customers will have.

Get it: https://spiredatasets.ee.iisc.ac.in/respin_corpus

How to study accents for ASR — the framework

Your mentor's question is really asking you to do three things:

Layer 1 — Understand the linguistics (study, not code)

Before writing any training script, you need to know *what* each accent does to Hindi phonemes so you can verify your model is actually learning the right thing. The key interference patterns for banking:

Numbers are the hardest. "Fifteen thousand" becomes:

- Bihari: "pandraa hazaar" (Bhojpuri numeral system)
- Punjabi: "pandhra hazar" (aspirates reduced)
- South Indian: "fifteen thowsand" (English borrowing, retroflex-free)

Your NER metric will catch this — but only if your eval set has accent-stratified samples.

Proper nouns mangle differently per accent:

- "CIBIL" → "sibil" (South Indian, no retroflex)
- "Aadhaar" → "aadhar" (UP, schwa deletion)
- "NACH" → "naach" (common confusion, means "dance" in Hindi)

Layer 2 — Accent-stratified evaluation (your contribution)

Run your existing eval pipeline but stratify results by accent. LAHAJA gives you this for free via its metadata CSV:

```
python

import pandas as pd

meta = pd.read_csv("lahaja/meta-data-lahaja.csv")

# Group by native language (proxy for accent)
accent_groups = {
    "Hindi-belt (UP/Bihar)": ["Bhojpuri", "Awadhi", "Maithili", "Magahi"],
    "Punjab/Haryana":      ["Punjabi", "Haryanvi"],
    "South India":         ["Tamil", "Telugu", "Kannada", "Malayalam"],
    "Bengal/East":         ["Bengali", "Odia", "Assamese"],
    "West India":          ["Marathi", "Gujarati"],
}

# Compute WER per accent group
for group_name, native_langs in accent_groups.items():
    subset = meta[meta['native_language'].isin(native_langs)]
```

```
# evaluate your model on this subset
# report WER separately per group
```

This becomes Table 2 in your blog — per-accent WER breakdown. The finding from LAHAJA paper is that models fail most on North-East and South India — if your fine-tuned model shows a different pattern (better on Hindi-belt because of Vaani training) that's a concrete result.

Layer 3 — Accent-aware fine-tuning

Use Vaani district data to build accent-stratified training. For banking in North India specifically, prioritize:

Priority 1 (highest customer density for banking):

- UP (Lucknow, Varanasi, Kanpur, Agra)
- Bihar (Patna, Muzaffarpur, Gaya)
- Delhi/NCR (Haryana substrate)
- Rajasthan (Jaipur, Jodhpur)

Priority 2 (significant but smaller):

- Punjab (Ludhiana, Amritsar)
- Madhya Pradesh (Bhopal, Indore)
- Maharashtra (Mumbai — for Marathi-Hindi mixing)

Priority 3 (South Indian callers — hardest accent, lower volume):

- Tamil Nadu, Karnataka, Andhra Pradesh

Your training data composition from Vaani should roughly reflect this priority order.

The updated data strategy including accents

Base training (general Hindi):

Kathbath (~500h subset) — already have this, accent diverse

Dialect/accent layer (NEW):

RESPIN finance subset — dialect-rich, finance domain, Bihar/UP heavy

Vaani Hindi-belt districts — spontaneous speech, geography-stratified

- Bihar: Patna, Gaya, Muzaffarpur, Bhagalpur
- UP: Lucknow, Varanasi, Kanpur, Agra, Meerut
- Punjab: Ludhiana, Amritsar
- Haryana: Rohtak, Gurugram

Domain layer (banking-specific):

MUCS Finance subset – closest existing domain data
Your synthetic Hinglish corpus – code-switching + banking terms

Evaluation only (never train on these):
LAHAJA – accent benchmark, per-accent WER breakdown
Your 100 banking WAVs – domain benchmark

How this changes your fine-tuning ablation

Add a fourth config to your existing A/B/

 Claude's response was interrupted.

Try again

continue

Resumed technical documentation on fine-tuning configurations >

How this changes your fine-tuning ablation (continued)

Add a fourth config to your existing A/B/C:

Config	Training Data	Primary Question
A	MUCS Finance only	Does domain data alone help?
B	Synthetic banking only	Does synthetic data help?
C	MUCS + Synthetic	Best domain combination
D	RESPIN Finance + Vaani Hindi-belt + MUCS + Synthetic	Does accent diversity + domain together win?

Config D is your headline result for the accent blog. The gap between Config C and Config D specifically measures the value of accent-diverse training — isolating exactly what your mentor asked you to study.

The three-benchmark evaluation structure

With accents in scope, you now have three evaluation axes that together tell a complete story:

Benchmark 1: Kathbath (3,151 samples)

- General Hindi, relatively clean, accent-mixed
- Already done for zero-shot. Re-run after each fine-tune config.
- Answers: "did fine-tuning hurt general Hindi?"

Benchmark 2: LAHAJA (12.5 hours, 83 districts)

- Multi-accent Hindi, extempore speech, has banking speaker metadata
- Run zero-shot first, then after Config C and Config D
- Answers: "which accents does each model struggle with?"
- Key table: WER broken down by accent group

Benchmark 3: Your 100 Banking WAVs

- Code-switched, domain-specific, your held-out test set
- Answers: "does this work for actual banking calls?"

The story across these three benchmarks is:

"Config C fixes the domain gap. Config D additionally fixes the accent gap. Neither hurts general Hindi significantly. Here's the per-accent breakdown showing which customer segments are now served reliably."

SLURM job additions for accent work

Two new jobs on top of your existing five:

Job 6: LAHAJA zero-shot evaluation (2 hrs)

Run this before any fine-tuning so you have the before/after comparison:

```
bash
```

```
#!/bin/bash
```

```
#SBATCH --job-name=lahaja_zeroshot
```

```
#SBATCH --gres=gpu:1
```

```
#SBATCH --mem=40G
```

```
#SBATCH --time=02:00:00
```

```
conda activate asr-eval
```

```
export CUDA_VISIBLE_DEVICES=1
```

```
export HF_HUB_OFFLINE=1
```

```
python -m banking_asr_eval.evaluate \
```

```
--manifest data/manifests/lahaja.json \  
--models indicwav2vec-hindi,voxtral,whisper-large-v3 \  
--output results/lahaja_zeroshot.csv \  
--stratify-by accent_group # new flag – reads from LAHAJA metadata CSV
```

Prepare the LAHAJA manifest with accent metadata:

```
python  
  
# prepare_lahaja.py  
import pandas as pd, json, os  
  
meta = pd.read_csv("lahaja/meta-data-lahaja.csv")  
  
# Map native languages to accent groups for stratified reporting  
accent_map = {  
    "Bhojpuri": "hindi_belt", "Awadhi": "hindi_belt",  
    "Maithili": "hindi_belt", "Magahi": "hindi_belt",  
    "Punjabi": "punjab_haryana", "Haryanvi": "punjab_haryana",  
    "Tamil": "south_india", "Telugu": "south_india",  
    "Kannada": "south_india", "Malayalam": "south_india",  
    "Bengali": "east_india", "Odia": "east_india",  
    "Marathi": "west_india", "Gujarati": "west_india",  
}  
  
manifest = []  
for _, row in meta.iterrows():  
    audio_path = f"lahaja/audio/{row['filename']}.wav"  
    if not os.path.exists(audio_path):  
        continue  
    manifest.append({  
        "audio_filepath": audio_path,  
        "text": row["transcript"],  
        "duration": row["duration"],  
        "accent_group": accent_map.get(row["native_language"], "other"),  
        "native_language": row["native_language"],  
        "occupation_domain": row["occupation_domain"],  
        "age_group": row["age_group"],  
        "gender": row["gender"],  
    })  
  
with open("data/manifests/lahaja.json", "w") as f:  
    for item in manifest:  
        f.write(json.dumps(item, ensure_ascii=False) + "\n")  
  
print(f"Manifest: {len(manifest)} samples")
```

```
# Print accent distribution
df = pd.DataFrame(manifest)
print(df.groupby("accent_group").size())
```

Job 7: Config D fine-tuning — accent-aware (10 hrs)

```
bash

#!/bin/bash
#SBATCH --job-name=indicwav2vec_configD
#SBATCH --gres=gpu:1
#SBATCH --mem=60G
#SBATCH --time=10:00:00

conda activate asr-eval
export CUDA_VISIBLE_DEVICES=1
export HF_HUB_OFFLINE=1

# Config D = RESPIN finance + Vaani Hindi-belt + MUCS + Synthetic
python finetune/indicwav2vec_finetune.py \
  --config D \
  --train-manifests \
    data/manifests/respin_finance_train.json \
    data/manifests/vaani_hindi_belt.json \
    data/manifests/mucs_finance_train.json \
    data/synthetic/manifest.json \
  --output /scratch/$USER/checkpoints/indicwav2vec-configD
```

Vaani manifest preparation — download district subsets:

```
python

# prepare_vaani.py – run on LOGIN NODE (needs internet)
from datasets import load_dataset
import json, os

# Hindi-belt districts prioritized for banking customer base
PRIORITY_DISTRICTS = [
  # Bihar
  ("Bihar", "Patna"), ("Bihar", "Gaya"), ("Bihar", "Muzaffarpur"),
  ("Bihar", "Bhagalpur"), ("Bihar", "Darbhanga"),
  # Uttar Pradesh
  ("UttarPradesh", "Lucknow"), ("UttarPradesh", "Varanasi"),
  ("UttarPradesh", "Kanpur"), ("UttarPradesh", "Agra"),
  ("UttarPradesh", "Meerut"), ("UttarPradesh", "Allahabad"),
```

```

# Punjab / Haryana
("Punjab", "Ludhiana"), ("Punjab", "Amritsar"),
("Haryana", "Rohtak"), ("Haryana", "Gurugram"),
# Rajasthan
("Rajasthan", "Jaipur"), ("Rajasthan", "Jodhpur"),
]

manifest = []
for state, district in PRIORITY_DISTRICTS:
    subset_name = f"{state}_{district}"
    try:
        ds = load_dataset(
            "ARTPARK-IISc/VAANI",
            subset_name,
            token=os.environ["HF_TOKEN"],
            split="train"
        )
        # Vaani is ~10% transcribed – filter to transcribed only
        transcribed = ds.filter(lambda x: x["transcription"] and len(x["transcri

    for sample in transcribed:
        manifest.append({
            "audio_filepath": sample["audio"]["path"],
            "text": sample["transcription"],
            "duration": sample["audio"]["sampling_rate"],
            "state": state,
            "district": district,
            "accent_group": "hindi_belt"
            if state in ["Bihar", "UttarPradesh"] else "punjab_haryana"
            if state in ["Punjab", "Haryana"] else "west_india",
        })
        print(f"Loaded {state}/{district}: {len(transcribed)} transcribed sample
    except Exception as e:
        print(f"Failed {state}/{district}: {e}")

with open("data/manifests/vaani_hindi_belt.json", "w") as f:
    for item in manifest:
        f.write(json.dumps(item, ensure_ascii=False) + "\n")

print(f"Total Vaani samples: {len(manifest)}")

```

The accent-stratified results table (what you're building toward)

This becomes the centerpiece table of your accent blog:

Per-accent WER breakdown on LAHAJA:

Accent Group	IndicWav2Vec zero-shot	IndicWav2Vec Config C	IndicWav2Vec Config D	Voxtral (best baseline)
Hindi-belt (UP/Bihar)	?	?	?	?
Punjab/Haryana	?	?	?	?
South India	?	?	?	?
East India (Bengal)	?	?	?	?
West India (Maharashtra)	?	?	?	?
Overall LAHAJA WER	?	?	?	?

The expected finding based on the LAHAJA paper and SandLogic's results:

- Config C will improve overall WER but may still struggle on Hindi-belt accents (because MUCS Finance and synthetic data don't capture dialect variation)
- Config D should specifically improve Hindi-belt and Punjab/Haryana rows because Vaani and RESPIN directly cover those districts
- South India will remain hardest regardless — this is your "future work" section

How to think about accents for your thesis narrative

Your mentor's accent question is pointing at something deeper than just "add more data." The conceptual framework to develop is:

Hindi is not one language acoustically. The phoneme inventory that a Bihari speaker uses when speaking Hindi is different from a Punjabi speaker's — not because of vocabulary, but because of phonological transfer from their mother tongue. An ASR model trained only on "standard" Hindi (newscaster Hindi, read speech from literate urban speakers) is implicitly trained on one socioeconomic and geographic slice of India.

For banking specifically this matters because:

- Debt collection calls skew toward lower-income customers — who are more likely to speak regional-accent Hindi
- Loan applications come from Tier 2/3 cities — again, heavier dialect influence
- KYC and onboarding calls include elderly customers — who have stronger regional accents and slower speech rates

This is your thesis argument: a production banking ASR system must be evaluated and fine-tuned with explicit accent stratification, not aggregate WER. A model with 15% overall WER but 40% WER on Bhojpuri-Hindi speakers is not a production-ready system for Indian banking.

Updated complete roadmap

Stage 1 – Baselines  Done

Kathbath zero-shot, banking synthetic zero-shot

Stage 2 – Accent baselines (NEW, do first)

Run zero-shot on LAHAJA, get per-accent WER breakdown

This gives you the "before" picture for accent work

Stage 3 – Domain fine-tuning

Config A, B, C (MUCS Finance + Synthetic)

Eval on banking 100 WAVs + LAHAJA

Stage 4 – Accent-aware fine-tuning (NEW)

Prepare Vaani Hindi-belt + RESPIN Finance manifests

Config D: all data combined

Eval on banking 100 WAVs + LAHAJA per-accent breakdown

Stage 5 – Error analysis

Per-accent error categories

Number errors by accent (Bihari number words vs standard Hindi)

Entity errors by accent

Stage 6 – Blog / thesis writing

Blog 6: "WER is not enough" → metrics paper

Blog 7: "Accents are not edge cases" → accent paper

Blog 8: "Fine-tuning for production banking" → synthesis

The accent work doesn't add much compute — it's mostly data preparation and evaluation. The Vaani download and RESPIN access take a day to set up on the login node. The Config D fine-tuning is one additional SLURM job. But the results it

produces — showing specifically which accents improve and by how much — elevate the work from "we fine-tuned a model" to "we built an ASR system that actually serves India's banking population."

so lahaja, vaani or respin?

Weighed dataset options against user context >

Each does a different job. They're not alternatives — they're three different roles:

LAHAJA → evaluation only, always

12.5 hours, 132 speakers, 83 districts — read and extempore speech across diverse accents, published at Interspeech 2024. It's a benchmark, not a training set. You run your models on it to *measure* accent robustness, not to train. Think of it like Kathbath but specifically for accent evaluation. Use it to fill your per-accent WER table.

[ResearchGate](#)

RESPIN → training, use this first

10,000+ hours, dialect-rich, covers Hindi, Bhojpuri, Chhattisgarhi, Magahi, Maithili — and crucially, has a finance domain subset specifically. This is your best training source because it combines two things you need simultaneously: dialect diversity AND finance domain. Start here before touching Vaani. [arxiv](#)

Vaani → training, use selectively if RESPIN isn't enough

~27,750 hours across 103 languages, district-level metadata — SandLogic filtered 366 hours of Hindi from 8 states and 51 districts and got 21% relative WER reduction on LAHAJA. The problem is only ~10% is transcribed, so usable volume is lower than it looks. More effort to prepare than RESPIN. [lith](#)

Concrete decision

If you have time for one dataset: RESPIN (finance domain + dialects, ready to use)

If you have time for two: RESPIN (train) + LAHAJA (eval)

If you have time for all three: RESPIN + Vaani Hindi-belt (train) + LAHAJA (eval)

Given your Param Rudra scheduling constraint — **start with RESPIN + LAHAJA**. That's one training source and one evaluation benchmark. Add Vaani in Config D only after you have Config C results and know whether the extra data is worth the preparation time.

so to clarify, i have done a zeroshot baseline of several asr on kathbath and 100 synthetic, then i did fine tune on synthetic transcript generated to tts synthetic dataset of 600 audio each of 5 seconds, now i am fine tuning on mucs+synthetic_600, when the models are done, i will evaluate on kathbath and sythetic 100. now i have to get baseline on lahaja, fine tune on respin and then eval on lahaja?

Yes, exactly right. Let me just clean that up into a clear linear sequence:

Your actual pipeline, in order

DONE 

- └─ Zero-shot baseline: Kathbath + synthetic_100
- └─ Fine-tune Run 1: synthetic_600 (TTS-generated)

IN PROGRESS 

- └─ Fine-tune Run 2: MUCS + synthetic_600
 - └─ Eval: Kathbath + synthetic_100

NEXT 

- └─ Step 1: Zero-shot baseline on LAHAJA (before any accent fine-tuning)
- └─ Step 2: Download + prepare RESPIN finance subset
- └─ Step 3: Fine-tune on RESPIN (+ optionally MUCS + synthetic_600)
- └─ Step 4: Eval on LAHAJA (per-accent WER breakdown)

One important clarification on Step 3

When you fine-tune on RESPIN, stack everything you have — don't start fresh:

```
Fine-tune Run 3 (accent-aware):
  RESPIN finance subset
+ MUCS finance
+ synthetic_600
→ Eval on Kathbath + synthetic_100 + LAHAJA
```

This way your comparison is clean:

Run	Training Data	Eval on LAHAJA
Zero-shot	—	← get this first (Step 1)
Run 1	synthetic_600	?
Run 2	MUCS + synthetic_600	?
Run 3	RESPIN + MUCS + synthetic_600	?

The delta between Run 2 and Run 3 on LAHAJA specifically isolates what RESPIN's accent diversity adds. That's your accent blog result.

One flag on your synthetic_600

$600 \times 5s = 50$ minutes of audio. That's quite small. It's enough to shift domain vocabulary but probably not enough to generalize. MUCS + RESPIN will do the heavy lifting — synthetic_600 contributes banking-specific terms and code-switching patterns that neither MUCS nor RESPIN have. Keep it in the mix but don't expect it to carry fine-tuning on its own, which is probably what you're already seeing in Run 1 results.



also what was musan used for then?

MUSAN is for **noise augmentation during training** — not a speech/transcript dataset at all.

It's a corpus of three types of noise:

- Background noise (rain, traffic, office hum)
- Music

- Babble (overlapping speech — most important for call center simulation)

The idea is you take your clean training audio (synthetic_600, MUCS, RESPIN) and mix MUSAN noise into it before feeding to the model:

```
python  
  
clean_audio + MUSAN_babble_noise → augmented training sample
```

This teaches the model to transcribe correctly even when there's background noise — which is exactly what a call center environment has.

Where it fits in your pipeline

```
Your clean training data (MUCS + RESPIN + synthetic_600)  
  ↓ apply MUSAN augmentation (during DataLoader)  
Noisy training samples → fine-tune model  
  ↓  
Model is now robust to call center noise  
  ↓  
Eval on LAHAJA + synthetic_100 (clean) — should still hold
```

Practically speaking

You don't apply MUSAN separately as a preprocessing step — you apply it on-the-fly during training using `audiomentations`. Each epoch sees a slightly different noisy version of the same clean audio, which acts as regularization too.

So in your current fine-tuning runs — if you haven't added MUSAN augmentation yet, that's something to add in Run 3 (RESPIN + MUCS + synthetic_600) rather than going back and redoing Run 1 and 2. Keep Run 1 and 2 as clean baselines, make Run 3 the augmented version. That way you can also measure the effect of noise augmentation separately from the effect of RESPIN data.

and which models should i fine tune? like i cant do on all..

Looking at your zero-shot results, the answer is clear — **two models only**:

Your zero-shot results tell you exactly which to pick

Model	Kathbath WER	Banking WER	RTF	Verdict
IndicWav2Vec-Hindi	11.64%	75.49%	0.09	Best Hindi, broken on banking
Voxtral-Mini-3B	17.66%	46.01%	0.54	Best banking, not fine-tuneable
Whisper-large-v3	28.82%	46.32%	1.49	Too slow, skip
Whisper-large-v3-turbo	32.01%	55.52%	0.50	Decent but skip
Whisper-medium	41.64%	179.17%	0.96	Broken but fixable

Fine-tune these two, nothing else

Model 1 — IndicWav2Vec-Hindi (highest priority)

The paradox model — best RTF (0.09, basically free compute), best Hindi WER, but collapses on code-switched banking. Fine-tuning on MUCS + RESPIN + synthetic_600 directly addresses the failure mode. If you fix this model, you have the best of everything — speed, Hindi quality, and banking domain. Highest upside of any fine-tuning experiment you can run.

Model 2 — Whisper-medium (second priority)

179% WER looks disqualifying but the 74% entity accuracy at zero-shot proves the architecture understands the content — it's just romanizing Hindi instead of transcribing it. Forcing Hindi transcription + fine-tuning on Hinglish data should drop it to 40-60% banking WER. Also the most documented fine-tuning path in HuggingFace ecosystem, so lowest risk of wasted SLURM slots.

Why not the others

Voxtral — can't fine-tune easily, it runs inside vLLM and is your strongest baseline. Keep it as the ceiling to beat, not a fine-tuning target.

Whisper-large-v3 / turbo — fine-tuning large models costs 3-4x more compute for marginal gains over medium. Your SLURM slots are too valuable.

Whisper-tiny — 254% WER on Kathbath, not worth it.

The narrative this creates

IndicWav2Vec fine-tuned: fastest model (RTF 0.09) + fixed domain gap
→ production recommendation if banking WER < Voxtral

Whisper-medium fine-tuned: was broken (179%) → now competitive
→ proves fine-tuning rescues even failed zero-shot models

If fine-tuned IndicWav2Vec beats Voxtral's 46% banking WER while being 6× faster — that's your thesis conclusion and krim.ai's deployment decision in one result.

IISc_RESPIN_train_bn_clean.tar.gz
Size: 81.11 GB

IISc_RESPIN_train_bn_seminoisys.tar.gz
Size: 14.39 GB

Show more

This is massive — don't download all of it. Here's exactly what to grab:

What the language codes mean

Code	Language	Relevance to banking Hindi
hi	Hindi	✓ Direct — download this
bh	Bhojpuri	✓ Bihar accent substrate — download this
mg	Magahi	✓ Bihar accent substrate — download this
mt	Maithili	✓ Bihar/UP accent substrate — download this

Code	Language	Relevance to banking Hindi
<code>bn</code>	Bengali	● Optional — Bengali-Hindi accent
<code>mr</code>	Marathi	● Optional — Maharashtra customers
<code>ch</code>	Chhattisgarhi	✗ Skip — low banking customer density
<code>kn</code>	Kannada	✗ Skip — South India, different problem
<code>te</code>	Telugu	✗ Skip — South India, different problem

Which quality tier to download

RESPIN has three tiers — clean, seminoisy, noisy. For fine-tuning ASR:

Download: clean + seminoisy only. Skip noisy.

- `clean` — high quality transcriptions, good for learning domain vocabulary
- `seminoisy` — some background noise, closer to real call conditions
- `noisy` — transcription quality degrades, will hurt more than help

Exact files to download

```
bash

# Priority 1 – Hindi (direct domain match)
IISc_RESPIN_train_hi_clean.tar.gz      # 63.81 GB
IISc_RESPIN_train_hi_seminoisy.tar.gz  # 24.51 GB
IISc_RESPIN_test_hi.tar.gz             # 303 MB  ← eval

# Priority 2 – Bihar accent substrates
IISc_RESPIN_train_bh_clean.tar.gz      # 81.40 GB  (Bhojpuri)
IISc_RESPIN_train_mg_clean.tar.gz      # 94.23 GB  (Magahi)
IISc_RESPIN_train_mt_clean.tar.gz      # 49.51 GB  (Maithili)

# Test sets for all (small, always get these)
IISc_RESPIN_test_bh.tar.gz             # 286 MB
IISc_RESPIN_test_mg.tar.gz             # 288 MB
IISc_RESPIN_test_mt.tar.gz             # 307 MB
```

Total if you follow this: ~313 GB — check your `$SCRATCH` quota first.

```
bash
```

```
df -h $SCRATCH
```

```
# need at least 400GB free (data + extracted + checkpoints)
```

If scratch quota is tight, minimum viable set

```
bash
```

```
# Absolute minimum for accent work:
```

```
IISc_RESPIN_train_hi_clean.tar.gz      # 63.81 GB – must have
```

```
IISc_RESPIN_train_hi_seminoisie.tar.gz # 24.51 GB – must have
```

```
IISc_RESPIN_train_bh_clean.tar.gz      # 81.40 GB – most important dialect
```

```
IISc_RESPIN_test_hi.tar.gz
```

```
IISc_RESPIN_test_bh.tar.gz
```

~170 GB total. This covers standard Hindi + the most common Bihar accent. Sufficient for one publishable result.

Download command for Param Rudra login node

```
bash
```

```
# do this on login node – has internet, compute nodes may not
```

```
cd /scratch/$USER
```

```
BASE_URL="https://spiredatasets.ee.iisc.ac.in/respincorpus"
```

```
# Priority 1: Hindi
```

```
wget -c $BASE_URL/IISc_RESPIN_train_hi_clean.tar.gz
```

```
wget -c $BASE_URL/IISc_RESPIN_train_hi_seminoisie.tar.gz
```

```
wget -c $BASE_URL/IISc_RESPIN_test_hi.tar.gz
```

```
# Priority 2: Bhojpuri (Bihar accent)
```

```
wget -c $BASE_URL/IISc_RESPIN_train_bh_clean.tar.gz
```

```
wget -c $BASE_URL/IISc_RESPIN_test_bh.tar.gz
```

```
# -c flag = resume if connection drops (important for 80GB files)
```

Run this inside a `screen` or `tmux` session — downloads this large will take hours and you don't want them dying when your SSH session drops:

```
bash
```

```
tmux new -s respin_download
# paste wget commands
# Ctrl+B then D to detach, tmux attach -t respin_download to recheck
```

After downloading — check the finance domain subset

RESPIN has domain metadata per utterance. Before fine-tuning, filter to finance domain only:

```
bash

# after extracting
tar -xzf IISc_RESPIN_train_hi_clean.tar.gz

# check the metadata structure
ls IISc_RESPIN_train_hi_clean/
# should have: audio/, transcripts/, metadata.json or similar
```

```
python

import json

# filter to finance domain utterances only
with open("IISc_RESPIN_train_hi_clean/metadata.json") as f:
    meta = json.load(f)

finance_samples = [
    s for s in meta
    if s.get("domain") == "finance"
]

print(f"Total samples: {len(meta)}")
print(f"Finance samples: {len(finance_samples)}")
# finance subset will be much smaller — that's your Config D training data
```

This finance-filtered subset is what goes into your fine-tuning manifest — not the full 63GB of everything.

